

EE7207 Neural Networks & Deep Learning

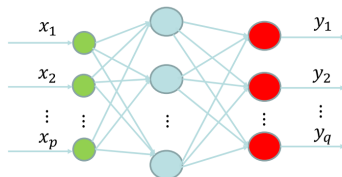
Multilayer Perceptron (MLP)

Jiawei Weng

jweng007@e.ntu.edu.sg

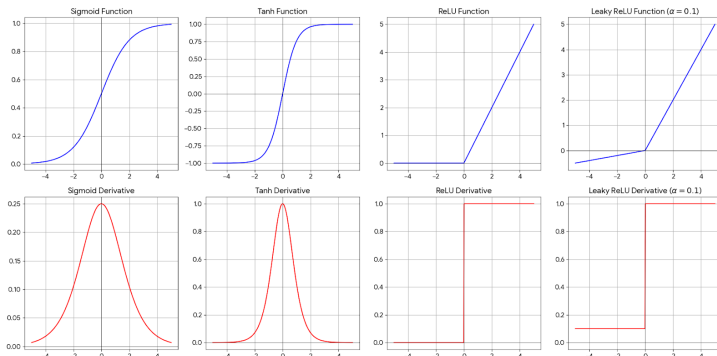
March 13, 2026

1. MLP: Architecture & Characteristics



- **Structure:** Input layer, one or more **hidden layers**, and an output layer.
- **Connectivity:** High degree of connectivity (typically **fully connected**).
- **Activation:** Use **smooth (continuous and differentiable)** functions like Sigmoid or Tanh.
- **Non-linearity:** Hidden neurons allow the network to learn complex, non-linear tasks.

Activation Function: Formulas & Derivatives)



Sigmoid

$$y = \frac{1}{1 + e^{-v}}$$

$$y' = y(1 - y)$$

Tanh

$$y = \frac{e^v - e^{-v}}{e^v + e^{-v}}$$

$$y' = 1 - y^2$$

ReLU

$$y = \max(0, x)$$

$$y' = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$$

Leaky ReLU

$$y = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}$$

$$y' = \begin{cases} 1 & x > 0 \\ \alpha & x < 0 \end{cases}$$

2. Supervised Learning Objective: Error Minimization

- **Error Signal:** For output neuron j at iteration n :

$$e_j(n) = d_j(n) - y_j(n)$$

- **Total Squared Error** (Cost Function):

$$E(n) = \frac{1}{2} \sum_{j=1}^q e_j^2(n)$$

- **Goal:** Find weight w_{ji} that minimizes $E(n)$ using **Gradient Descent**.
- **Delta Rule:** $\Delta w_{ji}(n) = -\eta \frac{\partial E(n)}{\partial w_{ji}(n)} = \eta \delta_j(n) y_i(n)$.

Key Formula for Exam Derivations

3. Back-propagation: The Local Gradient δ

The local gradient $\delta_j(n)$ depends on where the neuron is located:

Case 1: Output Layer Neuron

$$\delta_j(n) = e_j(n)\varphi'(v_j(n))$$

If Sigmoid is used: $\delta_j(n) = [d_j(n) - y_j(n)]y_j(n)[1 - y_j(n)]$.

Case 2: Hidden Layer Neuron

$$\delta_j(n) = \varphi'(v_j(n)) \sum_k \delta_k(n)w_{kj}(n)$$

The error is **back-propagated** from the next layer k .

Past Year Exam: 2022-2023-S1-Q3(a) Derivation

4. Training Modes & Momentum

- **Modes of Training:**

- ① **Sequential (On-line):** Update weights after each sample.
- ② **Batch:** Update weights after the entire epoch.
- ③ **Mini-batch:** Balance between the two.

- **Momentum (α):** Accelerates learning and avoids instability.

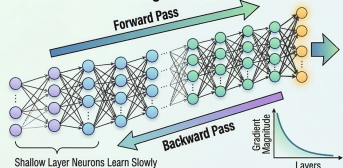
$$\Delta w_{ji}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_i(n)$$

- **Constraint:** $0 \leq \alpha < 1$.

5. The Vanishing Gradient Problem

Understanding Vanishing Gradient Problem in Deep Learning

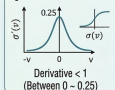
1. What is the Vanishing Gradient Phenomenon?



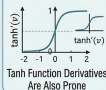
2. Causes: Chain Rule and Activation Functions

$$\text{Gradient} = \delta_{\text{out}} \cdot f'_L(v) \cdot w_L \cdot f'_{L-1}(v) \cdot w_{L-1} \dots \cdot f'_1(v) \cdot w_1$$

Sigmoid Derivative Function



Tanh Derivative Function



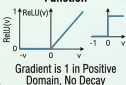
Multiplicative Effect (Network Deepening)

$0.2^{10} \approx 0$
 $0.2^{10} \approx 0 \Rightarrow 0.2^{10} \approx 0$
 $0.2^{15} \approx 0$
 $0.2^{15} \approx 0$

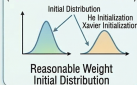
When Network is Too Deep, Products of Small Derivatives Exponentially Decrease

3. Solutions / Mitigation Methods

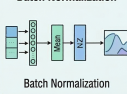
Use ReLU Activation Function



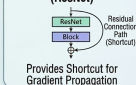
Parameter Initialization (He Initialization / Xavier Initialization)



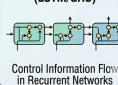
Batch Normalization



Use Residual Connections (ResNet)



Use Gated Structures (LSTM/GRU)



Past Year Exam: 2023-2024-S2-Q2(c) - Shallow vs Deep MLP

Past Year Quiz 1 - What is Vanishing Problem? Cause and Solutions?

3. (a) Consider the single-layer feedforward neural network shown in Figure 1 on page 3, where $x_1, x_2, \dots, x_p$ are the inputs, $w_1, w_2, \dots, w_p$ are the weights. The activation function of the output layer neuron is a sigmoid function. Assume the sequential learning is adopted, and the following loss function is used:

$$J = \frac{1}{2} [y(k) - d(k)]^2$$

where $y(k)$ and $d(k)$ are the output of the neural network and target value for the k^{th} sample. Derive the back-propagation weight updating rule.

(7 Marks)

Question: Derive the BP weight updating rule for a single-layer network with Sigmoid.

- **Step 1:** $J = \frac{1}{2}(y - d)^2$
- **Step 2:** Chain rule $\frac{\partial J}{\partial w_i} = \frac{\partial J}{\partial y} \frac{\partial y}{\partial v} \frac{\partial v}{\partial w_i}$
- **Step 3:** $\frac{\partial J}{\partial y} = (y - d) = -e$
 $\frac{\partial y}{\partial v} = y(1 - y)$
 $\frac{\partial v}{\partial w_i} = x_i$
- **Result:** $\Delta w_i = \eta e y (1 - y) x_i$

- (c) In deep neural network training, one frequently encountered problem is gradient vanishing. Discuss its impact and suggest solutions to the problem.

(6 Marks)

- **Impact:**

- Neurons in early layers learn much slower than later layers.
- Prevents the successful training of deep architectures.

- **Solutions:**

- 1 **ReLU Activation:** Derivative is 1 for $x > 0$, preventing gradient decay.
- 2 **Leaky ReLU:** Avoids "dead ReLU" by providing a small slope for $x < 0$.
- 3 **Batch Normalization:** Stabilizes the distribution of inputs to each layer.

Summary: MLP & BP Mind Map

